OBJECT ORIENTED PROGRAMMING
CCS20704, DCS20504, CCS23803

Received Date : 3 September 2026
Submission Due Date: 23 October 2026
Lecturer : Dr. Jamal Abdullahi
Weightage : 15%
Semester : SEPTEMBER 2026

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:		
I declare that: · ***This assignment is my/our own work.*** · ***I/we understand what is meant by plagiarism.*** · ***My lecturer has the right to deduct my marks in case of:*** - ***Late submission*** - ***Any plagiarism found in my assignment.***		
		
YEOP YAZID BIN YAHAYA **012026020796**	**SHUBAASHAN A/L BALAMURUGAN** **0120260206869**	**MUHAMMAD AIMAN BIN SULAIMAN** **012026020480**

Table Of Contents

Table Of Contents **2**

1. Introduction **3**

1.1 Overview of Object-Oriented Programming (OOP) 3

1.2 Assignment Objective 3

1.3 Chosen Object: E-Commerce Product 3

2.0. Discussion **4**

2.1 Application of Object-Oriented Programming Principles 4

2.2 Use of Selection and Looping Structures 5

3.0 Program Execution **6**

1. Introduction

1.1 Overview of Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is one of the basic programming paradigms focused on creating objects that package both the data (attributes) and the operations (methods). The main goal of OOP is to simulate real-life objects as software components in order to make the code reusable, modular and easy to maintain. The main concepts of OOP are Encapsulation, Inheritance, Polymorphism and Abstraction. Of these, one is encapsulation, which helps prevent direct access to an object's data, and allows only controlled access to the data, through public getters and setters, while providing security.

1.2 Assignment Objective

This assignment is aimed at giving students practice applying the basics of OOP – the emphasis will be on classes, objects and encapsulation – while creating a custom class to represent a real object. In this project, we will see how to use class attributes, default and parameterized constructors, how to access the data of a class and how to implement a custom operational logic using control structures within a java program.

1.3 Chosen Object: E-Commerce Product

The object selected for this assignment is an E-Commerce Product, a model of an item that is on offer in an online retail system. This object effectively collects qualitative and quantitative data, as there have been seven different attributes: productID using int, productName is the brand name using string, category is the retail classification using string, price is the monetary cost of the product using double, weightKg is the product's physical weight in Kg using double, warranty is the status of warranty using boolean, and isOnSale is a boolean flag indicating the product's promotional status. These attributes are encapsulated and remain hidden from the Product class, while custom methods are used to perform operations like calculating the ship eligibility based on product weight and promotional discount calculations.

2.0. Discussion

2.1 Application of Object-Oriented Programming Principles

The Ecommerce class system effectively demonstrates the core principles of Object-Oriented Programming (OOP), specifically focusing on classes, objects, encapsulation, constructors, and methods.

Encapsulation: All seven attributes (productID, productName, category, price, weightKG, warranty, and isOnSale) are declared as private. This ensures that external classes cannot directly modify the internal state of a product. Instead, controlled access and modification are handled via public getter and setter methods. Furthermore, input validation is incorporated into setters such as setPrice() and setWeightKG() using if conditions to ensure that invalid negative values cannot be assigned.

Classes and Objects: The Ecommerce class serves as the blueprint defining the properties and behaviors of an online retail item, while individual instances (such as item1 and item2) are instantiated in the Main class.

Constructors: Two constructors are implemented to provide flexibility when creating objects:

1. The default constructor initializes attributes with standard pre-defined values (e.g., product ID 1, name "Adibas", category "Shoes", price 100.00, weight 1.00, and warranty/sales flags set to true).
2. The parameterized constructor allows developers to pass custom arguments to initialize an object with specific product details instantly.

Custom Operational Methods: Three primary methods give the class functional behavior:

displayDetails(): Outputs all attributes of the product in a structured format.

calculateShippingFee(): Computes the total shipping cost based on the product's weight (weightKG * ratePerKG where ratePerKG is 5.00).

calculateSalePrice(): Evaluates whether isOnSale is active and applies a promotional discount calculation when applicable.

2.2 Use of Selection and Looping Structures

Control structures are integrated into both the Ecommerce class and the Main execution class to manage program logic and data processing interactively:

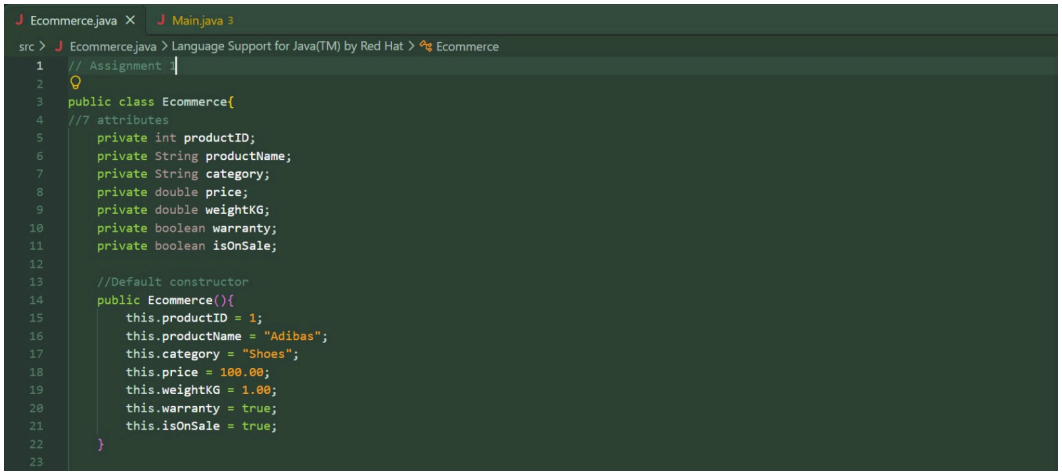
Selection Structures (if statements):

- Inside the setter methods (setPrice and setWeightKG), if conditions validate incoming data to prevent negative values.
- Inside the calculateSalePrice() method, an if structure checks if isOnSale is true before deducting a 10.00% discount from the base price.

Looping Structures (for loop):

- In the Main class, a standard for loop is utilized to loop through an array of Ecommerce objects representing a shopping cart (Ecommerce[] cart = {item1, item2};).
- The loop dynamically traverses the array, using getter methods (getProductName(), getCategory(), and getPrice()) to neatly print out a numbered summary list of all items in the cart along with their prices.

3.0 Program Execution

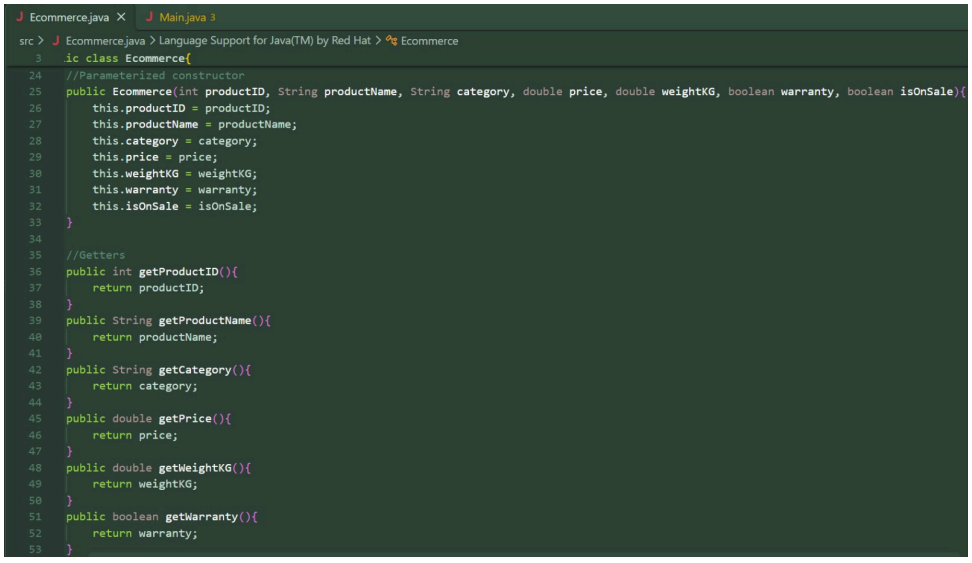


```java
// Assignment 3

public class Ecommerce{
//7 attributes
    private int productID;
    private String productName;
    private String category;
    private double price;
    private double weightKG;
    private boolean warranty;
    private boolean isOnSale;

    //Default constructor
    public Ecommerce(){
        this.productID = 1;
        this.productName = "Adibas";
        this.category = "Shoes";
        this.price = 100.00;
        this.weightKG = 1.00;
        this.warranty = true;
        this.isOnSale = true;
    }
```

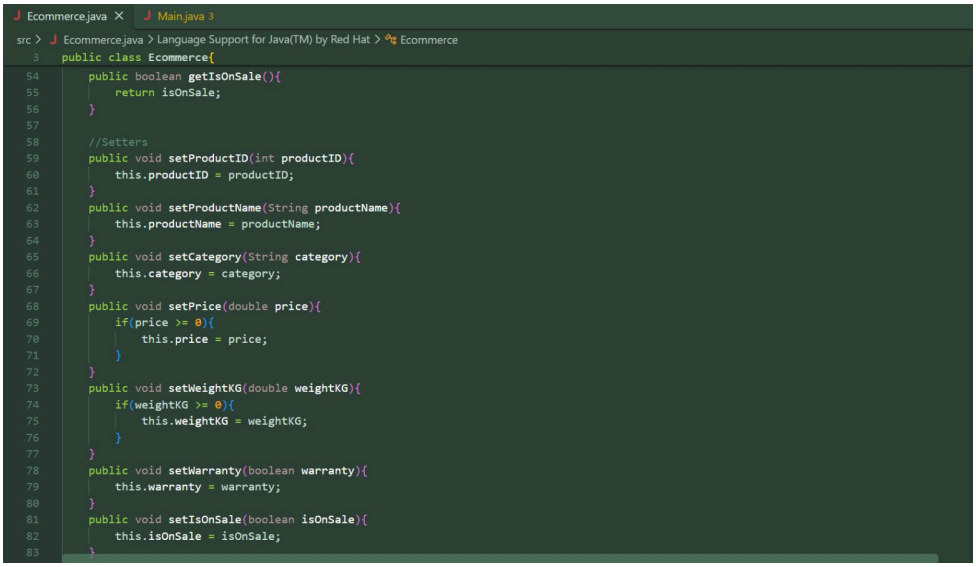
Figure 3.1 Declaration of private attributes and default constructor for the Ecommerce class.



```java
//Parameterized constructor
public Ecommerce(int productID, String productName, String category, double price, double weightKG, boolean warranty, boolean isOnSale){
    this.productID = productID;
    this.productName = productName;
    this.category = category;
    this.price = price;
    this.weightKG = weightKG;
    this.warranty = warranty;
    this.isOnSale = isOnSale;
}

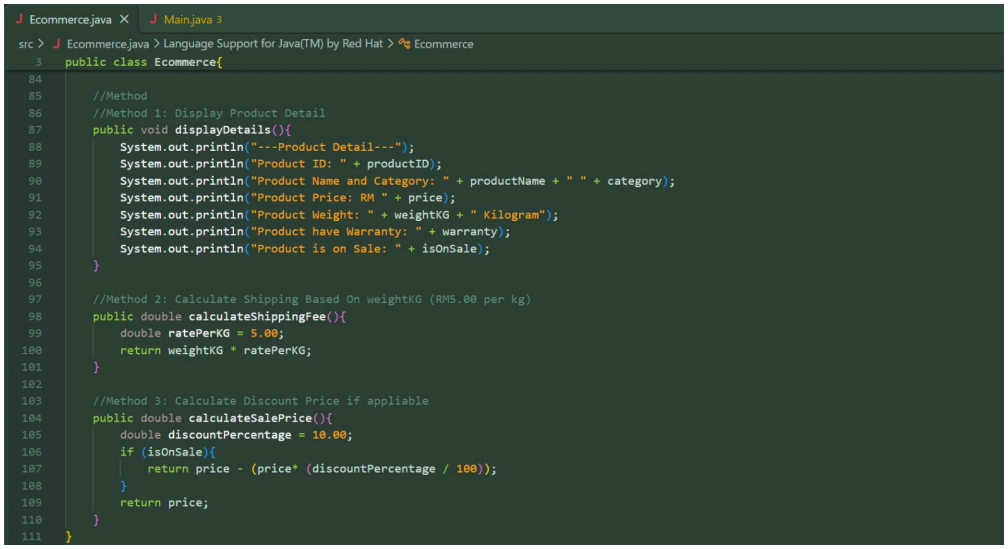
//Getters
public int getProductID(){
    return productID;
}
public String getProductName(){
    return productName;
}
public String getCategory(){
    return category;
}
public double getPrice(){
    return price;
}
public double getWeightKG(){
    return weightKG;
}
public boolean getWarranty(){
    return warranty;
}
```

Figure 3.2 Implementation of the Parameterized Constructor and Getter Methods.



```
Ecommerce.java   Main.java 3
src > Ecommerce.java > Language Support for Java(TM) by Red Hat > Ecommerce
3   public class Ecommerce{
54      public boolean getIsOnSale(){
55          return isOnSale;
56      }
57
58      //Setters
59      public void setProductID(int productID){
60          this.productID = productID;
61      }
62      public void setProductName(String productName){
63          this.productName = productName;
64      }
65      public void setCategory(String category){
66          this.category = category;
67      }
68      public void setPrice(double price){
69          if(price >= 0){
70              this.price = price;
71          }
72      }
73      public void setWeightKG(double weightKG){
74          if(weightKG >= 0){
75              this.weightKG = weightKG;
76          }
77      }
78      public void setWarranty(boolean warranty){
79          this.warranty = warranty;
80      }
81      public void setIsOnSale(boolean isOnSale){
82          this.isOnSale = isOnSale;
83      }
```

Figure 3.3 Implementation of the Setter Methods.



```
Ecommerce.java   Main.java 3
src > Ecommerce.java > Language Support for Java(TM) by Red Hat > Ecommerce
3   public class Ecommerce{
84
85      //Method
86      //Method 1: Display Product Detail
87      public void displayDetails(){
88          System.out.println("---Product Detail---");
89          System.out.println("Product ID: " + productID);
90          System.out.println("Product Name and Category: " + productName + " " + category);
91          System.out.println("Product Price: RM " + price);
92          System.out.println("Product Weight: " + weightKG + " Kilogram");
93          System.out.println("Product have Warranty: " + warranty);
94          System.out.println("Product is on Sale: " + isOnSale);
95      }
96
97      //Method 2: Calculate Shipping Based On weightKG (RM5.00 per kg)
98      public double calculateShippingFee(){
99          double ratePerKG = 5.00;
100         return weightKG * ratePerKG;
101     }
102
103     //Method 3: Calculate Discount Price if appliable
104     public double calculateSalePrice(){
105         double discountPercentage = 10.00;
106         if (isOnSale){
107             return price - (price* (discountPercentage / 100));
108         }
109         return price;
110     }
111 }
```

Figure 3.4 Implementation of Custom Operational Methods (displayDetails, calculateShippingFee, and calculateSalePrice).

```java
//Assignment 1

public class Main{
    Run main | Debug main | Run | Debug
    public static void main(String[] args){
        //Test Default Constrictor
        Ecommerce item1 = new Ecommerce();

        //Test Parameterized Constrictor
        Ecommerce item2 = new Ecommerce(productID: 2, productName: "Bolo", category: "Shirt", price: 200.00, weight… 0.25, true, false);

        //Display Product Detail with Method
        //Item 1 = Default
        System.out.println("---Item 1 Details---");
        item1.displayDetails();
        System.out.println("Shipping Fee: RM " + String.format("%.2f", item1.calculateShippingFee()));
        System.out.println("Price after 10% sale discount: RM " + String.format("%.2f", item1.calculateSalePrice()));
        //Item 2 = Paramiterized
        System.out.println("\n---Item 2 Details---");
        item2.displayDetails();
        System.out.println("Shipping Fee: RM " + String.format("%.2f", item2.calculateShippingFee()));
```

Figure 3.5 Implementation of the main class to test instance of an object, custom methods and calculation

```java
        //Attribute manipulation
        System.out.println("\n---Updating Item 2 Price---");
        item2.setPrice(price: 150.00);
        System.out.println("Updated Price for " + item2.getProductName() + " " + item2.getCategory() + "is: RM " + String.format("%.2f", item2.getPrice()));

        //Looping Structure (For Statement in Array) (Shopping Cart Summary)
        Ecommerce[] cart = {item1, item2};
        System.out.println("\n---Shopping Cart Summary---");
        for (int i = 0; i < cart.length; i++){
            System.out.println((i + 1) + ". " + cart[i].getProductName() + " " + cart[i].getCategory() + "- RM " + cart[i].getPrice());

            //Selection Structure (If Statement) (Discount Status)
            if (cart[i].getIsOnSale()){
                System.out.println("Congratulations! You have got a great deal for the item.");
            }else{
                System.out.println("Regular price item.");
            }
        }
    }
}
```

Figure 3.6 Implementation of Looping and Selection Structures for Shopping Cart Processing.

Link for the source code in GitHub:
https://github.com/AimanSulaiman169/EcommerceAssignment1-OOP